

# HW06 — AI-First API Testing Report

## Submission identity

- Student: Nguyễn Đình Thái Hưng
- Student ID: 23127373
- Repository: <https://github.com/z3nz3nn/HW06-software-testing> — PRIVATE until the student approves the final public step
- SUT: EShop at <http://127.0.0.1:3000>
- Evidence status: official local Newman run with X-Student-Id: 23127373

## 1. API selection and collision check

| Pool | Requirement | Chosen endpoint                 | Collision check                   |
|------|-------------|---------------------------------|-----------------------------------|
| A    | FR-03       | POST /api/reset-password        | Not selected by any listed friend |
| B    | FR-09       | POST /api/apply-coupon          | Not selected by any listed friend |
| C    | FR-16       | POST /api/admin/import-products | Not selected by any listed friend |

The selection deliberately avoids all nine endpoints reported by the three friends: login, checkout, admin order status, product listing, cart, admin coupon creation, registration, order cancelation, and category creation.

## 2. Method

For each endpoint I used a phased AI-first workflow: requirements decomposition → ambiguity correction → candidate generation → human audit → at least five student-added cases → data-driven Postman/Newman execution. Gemini was not allowed to inspect the implementation before generating candidates. Follow-up prompts explicitly corrected invented assumptions and unusable oracles. Full verbatim evidence is indexed in ai-audit.md.

The executable oracle distinguishes normative rules from unspecified behavior. ACCEPT/APPLY/COMMIT and REJECT/ROLLBACK cases assert requirements. OBSERVE cases assert only safety/stability and remain marked INCOMPLETE when the specification does not permit a binary verdict.

## 3. Test inventory and official execution result

| API                             | AI candidates | VALID | INVALID | INCOMPLETE | Student-added | Executed | Passed | Failed |
|---------------------------------|---------------|-------|---------|------------|---------------|----------|--------|--------|
| POST /api/reset-password        | 42            | 32    | 0       | 10         | 6             | 48       | 35     | 13     |
| POST /api/apply-coupon          | 44            | 33    | 2       | 9          | 5             | 49       | 36     | 13     |
| POST /api/admin/import-products | 56            | 32    | 7       | 17         | 5             | 61       | 42     | 19     |

The official 158-case run used X-Student-Id: 23127373. Its 113 passing and 45 failing cases are not fabricated “green” evidence: failures are preserved because they reveal requirement violations.

## 4. Coverage

| API             | Domain partitions                | State transitions                           | Security                                          | Schema validation                                        |
|-----------------|----------------------------------|---------------------------------------------|---------------------------------------------------|----------------------------------------------------------|
| reset-password  | email, resetToken, newPassword   | OTP active/reissued/old/used/email-bound    | SEC-01, SEC-05, SEC-07; SQLi; leakage             | Malformed/missing/null/type/extra fields                 |
| apply-coupon    | code, total_amount, user_id, JWT | active/expired/disabled/usage threshold     | SEC-02; missing/invalid/expired JWT; IDOR         | Required numeric fields, formula and error stability     |
| import-products | auth and every product property  | commit/rollback, mixed batches, concurrency | FR-12/SEC-03 role gate; SQL/XSS literals; leakage | Top-level and row schema, malformed JSON, duplicate keys |

The reset suite covers 42 AI candidates plus 6 extensions. Coupon uses 45 generated candidates, removes one invalid concurrency candidate, corrects three invalid candidates, and executes 44 corrected AI rows plus 5 extensions. Import uses 48 initial candidates plus 8 corrective supplemental candidates and 5 extensions. This keeps all three endpoints above the  $\geq 35$  target after audit.

## 5. Human audit decisions

- VALID: a traceable requirement and executable oracle exist.
- INVALID: the proposed case cannot establish its precondition or contradicts the endpoint contract; it is retained in the audit ledger but excluded or converted before execution.
- INCOMPLETE: useful exploratory input, but the specification leaves the expected result ambiguous; assertions are conservative.
- STUDENT\_ADDED: a security, protocol, metamorphic, state, or white-box risk missed by Gemini.

Every row contains its label and reason in the Excel workbook and canonical case-definitions.mjs. Manual student sign-off remains required before submission; automated generation is not represented as personal review.

## 6. Postman/Newman implementation

Used features: collection folders; collection/environment/local variables; pre-request scripts; automatic X-Student-Id injection; dynamic unique users; bearer-token variables; data-driven JSON iterations; request chaining; setup/teardown fixtures; response-schema/formula/security assertions; CLI, JSON, JUnit, and HTML reporters. The local fixture API is limited to otherwise unreachable preconditions such as exact OTPs, coupon states, usage counts, and database snapshots. It is not the target API.

Not claimed: cloud workspace collaboration, Postman Monitor, or a hosted mock server. They are optional examples in the brief and were not necessary for deterministic local execution.

Reproduce from PowerShell in the repository root:

```
npm.cmd install
npm.cmd run generate:postman
npm.cmd run test:api -- --student-id 23127373
npm.cmd run verify:artifacts
```

## 7. Genuine defects

| ID     | Severity | Area           | Finding                                                                                                               | Evidence cases |
|--------|----------|----------------|-----------------------------------------------------------------------------------------------------------------------|----------------|
| BUG-01 | Critical | Authentication | Passwords are stored in plaintext and login returns the full user record including password/reset_token/locked_until. | R-H-05, R-H-06 |

| ID     | Severity | Area                 | Finding                                                                                                                           | Evidence cases                    |
|--------|----------|----------------------|-----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| BUG-02 | High     | Password reset       | Reset accepts weak, missing, null, empty, and numeric passwords, contrary to FR-01.                                               | R-AI-05..10, R-AI-35..38          |
| BUG-03 | High     | OTP lifecycle        | Forgot-password generates only four digits and stores no expiry timestamp; SEC-07 cannot be met.                                  | Source review + R-AI-15           |
| BUG-04 | Critical | Coupon authorization | Apply-coupon has no JWT gate and trusts body user_id, enabling identity substitution and usage-limit bypass.                      | C-AI-02..06, C-AI-08..09          |
| BUG-05 | High     | Coupon formula       | Percent discount uses $\text{total} \times (1 - \text{discount\_value})$ , yielding negative discounts and inflated final totals. | C-AI-01, C-AI-19, C-AI-34         |
| BUG-06 | Medium   | Coupon boundary      | Minimum-order check uses $>$ instead of $\geq$ , rejecting exact threshold values.                                                | C-AI-18, C-AI-21, C-AI-22         |
| BUG-07 | Critical | Admin authorization  | Import-products authenticates a JWT but never enforces <code>role=admin</code> .                                                  | I-AI-02, I-AI-56                  |
| BUG-08 | High     | Import validation    | Rows validate only truthy name; invalid price/category/name-length values are inserted.                                           | I-AI-21..33, I-H-04               |
| BUG-09 | High     | Import atomicity     | Mixed-invalid batches partially commit instead of rolling back the whole batch.                                                   | I-AI-40..42, I-AI-51, I-H-01      |
| BUG-10 | Medium   | Error handling       | Malformed JSON and null rows expose Express/Node stack details.                                                                   | R-AI-40, C-AI-40, I-AI-45, I-H-01 |

Detailed reproduction steps, expected/actual results, and evidence references are in bug-reports.md. Source inspection supports root-cause analysis; the Newman failures remain the black-box evidence.

## 8. CI/CD

The workflow in api-tests.yml installs dependencies, regenerates artifacts, starts the local SUT, runs Newman, and uploads reports. The required all-pass versus one-fail pair is implemented as an explicit mutation-demonstration lane, separate from the diagnostic lane that preserves real SUT failures. The run links, commit IDs, and screenshots are recorded in ci-cd-report.md; public repository visibility remains a separate final submission gate.

## 9. Agent Skill

The reusable skill under `skills/eshop-api-test-generator` decomposes specifications, classifies normative versus observational oracles, generates  $\geq 35$  candidates, audits them, adds human-risk prompts, and emits canonical rows/Postman data. Pseudocode is provided. An editable Mermaid scaffold is available in `agent-skill-diagram.mmd`; the student must review or revise it and export the final `agent-skill-diagram.png` before submission.

## 10. Limitations and mandatory human review

OTP expiry and true concurrency need controlled time/parallel facilities not exposed by the published API. Identity, the official Newman rerun, ten issue records, CI run-pair evidence, the decision to omit the optional video, and the 100/100 self-assessment are recorded. The student must still personally review/sign off rows, take the non-fabricated console screenshot, spot-check the Newman HTML, confirm the AI audit, draw the diagram, approve the final repository-visibility change, inspect the final ZIP, and submit it on Moodle. The synchronized machine-readable state is `submission-status.json`; see `manual-checklist.md` for the human actions.

## 11. AI Critique (267 words)

Gemini was productive at decomposing parameters and proposing broad candidate pools, but its first answers showed why AI output cannot be used as an oracle without review. For password reset, it introduced RFC 5322 as if the specification required it, implied unlisted special characters were forbidden, and invented exact status-code expectations. For coupons, it assumed applying a coupon increments usage even though the endpoint only calculates a discount; it also under-specified the trust boundary between the JWT identity and the body `user_id`. For product import, it mixed frontend CSV-parser concerns with the JSON API, treated SQL-looking literals as invalid input, and proposed fault-injection and concurrency cases without executable preconditions. These failures came from three causes: the model filled specification gaps with common conventions, optimized for an impressive list rather than testability, and did not initially distinguish a normative requirement from an observational question.

Follow-up prompts improved the result because they named each faulty assumption and demanded a correction ledger. I retained incomplete cases only when their assertions could be limited to stability, security, or state observation; invalid cases were removed or replaced. The student-added cases then targeted null rows, authorization before processing, identity substitution, information disclosure, protocol behavior, and atomic rollback—risks the generic generation phase missed.

The main lesson is that collaboration with AI needs an evidence hierarchy: authoritative requirements first, explicit ambiguity second, executable preconditions third, and implementation observations last. A large case count is not quality by itself. Each case needs traceability, a defensible oracle, isolation, and a recorded audit decision. AI is best used as a fast hypothesis generator; responsibility for correctness remains human.

The standalone Markdown source is `ai-critique.md`; this full copy is included so the mandatory critique is also present in the Main Report PDF.

## 12. AI declaration

I use AI tools for requirements analysis, test-candidate generation, critique, automation assistance, and report scaffolding. Full prompts, outputs, timestamps, corrections, and evidence paths are disclosed. Final correctness and submission responsibility remain with the student.